Vibe code with me

Hi Claude! I'm new here. Could you vibe code with me? If you need more information from me, ask me 1-2 key questions right away. If you think I should give you more context or upload anything to help you do a better job, let me know.

If it makes sense, create something we can look at together—like a visual, a checklist, or something interactive. Thanks for your help!

Hey, welcome! Excited to build something with you 🎉 Let's start here:

Q: What are you looking to build?
A: [No preference]

Love it — I'll surprise you then! Let me build something fun and interactive that shows off what we can do together.

Reading frontend design skill >

I'll build something fun, interactive, and surprising — a **mood-based generative poem machine**. Pick a mood, get a beautiful AI-written poem. Let me make it gorgeous.

Mood-based AI poem generator with stunning visuals >

Claude's response was interrupted.

Hello, I need help making a public transport system in python.

The system is supposed to make it easier for passengers to use buses and trains. There are three types of users: Regular commuter, Student and Senior. There are also different types of tickets: single, period and

bundles (multiple single tickets that activate at the same time). There may be different offers for students and seniors.

The system should also be able to handle wrong inputs (for example invalid amount of tickets).

The system is needed to claculate the ticket prices correctly and consistently, even when many users are active at the same time.

Make the feature, no UI, no unit tests.

Check available skills >

No relevant skill needed here — this is pure Python. Let me design a clean, well-structured system.

Key design decisions:

- **Enums** for user types and ticket types (safe, no magic strings)
- **Dataclasses** for immutable pricing config
- **Thread-safe** price calculation using locks (handles concurrent users)
- **Custom exceptions** for invalid inputs
- **Strategy pattern** for discount logic per user type

Created 4 files, ran a command >

All 20 concurrent purchases succeeded, all 8 error cases raised the right exceptions. Let me copy to outputs.

Ran a command, read a file >

Here's what was built — 4 files with a clean separation of concerns:

`models.py` — All enums (`UserType`, `TicketType`, `TransportMode`), custom exceptions, and immutable frozen dataclasses for config and results.

`pricing.py` — The `PricingEngine`. Thread-safe via `RLock`. Handles all three ticket types, resolves and validates all inputs, and applies user discounts correctly. Bundles stack both the bundle-tier discount *and* the user discount multiplicatively (so a student buying a 10-pack gets both benefits).

`service.py` — `TicketingService`, the public façade. Has a general `purchase()` method plus `single()`, `period_pass()`, and `bundle()` shortcuts for cleaner call sites.

`config.py` — The default pricing config (easily replaced with a DB-loaded one), plus a runnable demo (`python config.py`) showing all ticket types, all user types, all error cases, and 20 concurrent threads.

A few design choices worth knowing about:

- **Discounts are fully data-driven** — changing a student discount is one number in `config.py`, no logic changes needed.
- **Bundle best-fit** — if a customer requests a bundle size that doesn't exactly match a tier, the engine picks the largest tier that fits, so you never have to define every possible size.
- **All prices are rounded at the final step** only, to avoid floating-point drift accumulating across intermediate calculations.

> Perfect, I would also like that the personal and payment information is protected if it isn't already.
>
> The bundles should not be just specific amounts, the amount should be chosen by the passengers, maybe there could be a max, and of course not any negative amounts.
>
> Now I just want to make sure that everything is implemented, if it's
>
> Show more

Let me audit the current code against every requirement before touching anything.

Viewed 2 files >

Good, now I have a clear picture. Here's my gap analysis before writing a single line:

Already solid: ✅ Three user types + discounts, ✅ bus/train/single/period/bundle, ✅ thread-safe engine, ✅ input validation, ✅ immutable config.

Gaps to fix:

1. **Personal & payment data protection** — completely absent. Need a `PassengerProfile` with encrypted PII and a `PaymentCard` that only stores a masked pan + token, never the raw number.
2. **Free-form bundle size** — currently bundles snap to configured "tiers". Need to remove that and let passengers choose any size within min/max bounds.
3. **Mixed-passenger bundles** — a bundle currently has one `user_type`. Need a `BundleItem` concept so one bundle purchase can contain e.g. 3 regular + 2 student tickets and price each correctly.

Ran 7 commands, created a file >

Everything passes. Copying the four files to outputs.

Ran a command, read a file >

Here's a summary of what changed and why, against each requirement:

New: `security.py`

- `PassengerProfile` — name, email, and date of birth are encrypted with **Fernet (AES-128-CBC + HMAC-SHA256)** the moment they're set. Reading a field decrypts on the fly. `repr` / `str` are redacted so PII can never leak into logs.
- `PaymentCard` — raw card numbers are validated (length + **Luhn checksum**), then immediately discarded. Only the last four digits and a PSP-issued payment token are stored. Cardholder name is encrypted at rest.
- The encryption key is loaded from the env-var `TRANSPORT_PII_KEY`; in production this should come from a secrets manager (AWS KMS, Vault, etc.). On first run without the key set, it prints a one-time generated key you can export.

Bundles are now free-form — `BundleConfig` tiers are gone entirely. Passengers pick any size between `min_bundle_size` (2) and `max_bundle_size` (100). Negative and zero values are rejected with a clear error.

Mixed-passenger bundles — a new `BundleItem(user_type, count)` dataclass lets one bundle purchase contain different passenger types. Each line is priced at that user's personal discount plus the flat bundle discount, stacked multiplicatively. The result is a `BundlePriceBreakdown` with an itemised line per passenger type.

Everything that was already working (three user types, bus/train, single/period, thread safety, all error cases) is unchanged.